Guida: OmniRoute e OpenCode in locale su un PC Windows

Questa guida porta da un PC Windows vuoto a un agente di programmazione che funziona: OpenCode (l'agente) collegato a OmniRoute (il centralino che sceglie il fornitore del modello). Si procede a strati: ogni strato si prova da solo prima di passare al successivo. Se qualcosa non va, si sa subito in quale strato guardare.

Nomi, chiavi e percorsi nella guida sono segnaposto: `<chiave>` indica una chiave che generate voi, `%USERPROFILE%` è la vostra cartella utente.

Prima di cominciare

Cosa serve

- Windows 10 o 11, con PowerShell (è già installato).
- Node.js: OmniRoute richiede la versione 22.22.2 o successive della 22, oppure la 24 o successive. Si controlla con `node -v` in PowerShell. Se manca, si scarica la versione LTS da nodejs.org.
- Una chiave API di almeno un fornitore di modelli. In questa guida l'esempio è Anthropic; la chiave si crea nella console del fornitore e si paga a consumo.
- Circa mezz'ora la prima volta.

Come funziona, in due righe. OpenCode non parla direttamente con il fornitore del modello: parla con OmniRoute, che gira sul vostro stesso PC all'indirizzo `http://localhost:20128`. OmniRoute inoltra la richiesta al fornitore che avete configurato e riporta la risposta. L'indirizzo `localhost` significa "questo computer": nulla esce dal PC finché OmniRoute non contatta il fornitore.

Parte 1. Installare OmniRoute

Metodo consigliato: npm

In PowerShell:

```
npm install -g omniroute
```

`-g` installa il programma per tutto l'utente, non solo nella cartella corrente. A fine installazione il comando `omniroute` è disponibile in ogni terminale nuovo.

Se compare un avviso sugli script di installazione bloccati (succede con npm 11 e successivi), OmniRoute ha bisogno che il suo script finale venga eseguito. In quel caso si rilancia l'installazione autorizzando i pacchetti che npm elenca nell'avviso. La forma indicata dalla documentazione di OmniRoute è:

```
npm install -g --allow-scripts=omniroute,keytar,onnxruntime-
node,@parcel/watcher,@swc/core,protobufjs,esbuild omniroute
```

Per controllare la versione installata: `npm ls -g omniroute`. Al 28 ottobre 2026 l'ultima versione pubblicata è la 3.8.52.

Alternativa: Docker

Serve Docker Desktop già installato e avviato. Il comando della documentazione è:

```
docker run -d --name omniroute --restart unless-stopped --stop-timeout 40 -p
127.0.0.1:20128:20128 -v omniroute-data:/app/data diegosouzapw/omniroute:latest
```

Il pezzo `127.0.0.1:20128:20128` limita l'accesso a questo PC. I dati stanno nel volume `omniroute-data`. Con Docker le variabili d'ambiente (password, chiavi) si passano con l'opzione `-e NOME=valore`.

Alternativa: dal codice sorgente

Serve Git. Conviene solo se volete modificare il programma.

```
git clone https://github.com/diegosouzapw/OmniRoute.git
cd OmniRoute
copy .env.example .env
npm install
$env:PORT = "20128"; npm run dev
```

Attenzione: `npm run dev` è la modalità di sviluppo. Funziona finché il terminale resta aperto; chiuso quello, OmniRoute si spegne. È il motivo per cui a molte persone "OmniRoute non risponde" (vedi gli errori frequenti).

Parte 2. Prima accensione e sicurezza

La password del pannello e l'obbligo della chiave

Due impostazioni vanno decise prima del primo avvio:

Variabile	Cosa fa	Cosa mettere
`INITIAL_PASSWORD`	Password del pannello di controllo, usata solo alla prima accensione, finché nessuna password è salvata. Il valore predefinito del modello di configurazione è `CHANGEME`, che è pubblico: chiunque lo conosce.	Una password vostra, lunga. In alternativa, lasciarla vuota e crearla nella procedura guidata del pannello.
`REQUIRE_API_KEY`	Se è `true`, ogni richiesta alla parte `/v1/` (quella che usano gli agenti) deve portare una chiave OmniRoute. Se è `false`, chiunque raggiunga la porta può usare i vostri fornitori e quindi spendere i vostri soldi.	`true`

Secondo la documentazione, `REQUIRE_API_KEY=false` va bene solo se OmniRoute è raggiungibile unicamente da questo PC o se davanti c'è un sistema che controlla gli accessi. Per prudenza si imposta `true` comunque. Un dettaglio: la documentazione precisa che la richiesta `GET /v1/models` segue il controllo di accesso del pannello e non questa impostazione, quindi un rifiuto lì non dice nulla sulla protezione delle richieste vere.

Conta anche l'indirizzo su cui OmniRoute ascolta. Con le impostazioni predefinite ascolta su tutte le interfacce di rete (`0.0.0.0`): chi è nella stessa rete può raggiungerlo. Per tenerlo chiuso dentro il PC, con il comando `omniroute serve` si usa la variabile `OMNIROUTE_SERVER_HOST=127.0.0.1`. Non usate la variabile `HOSTNAME`: il sistema la riempie con il nome del computer.

Con npm, il modo più sicuro di passare le variabili è scriverle nella finestra PowerShell prima di avviare. Valgono solo per quella finestra e non lasciano file in giro:

```
$env:INITIAL_PASSWORD = "<una-password-lunga-vostra>"
$env:REQUIRE_API_KEY = "true"
```

```
$env:OMNIROUTE_SERVER_HOST = "127.0.0.1"
omniroute serve
```

Se installate da sorgente o con Docker, le stesse variabili si scrivono nel file `.env` (sorgente) o con `-e` (Docker).

> Da verificare a schermo: in quale cartella un'installazione npm cerca il file `.env`. La documentazione letta non lo dice. Il comando `omniroute env show` (se presente nella vostra versione) mostra i valori in uso. Secondo la documentazione, però, un'installazione npm crea un file `.env` copiando il modello di configurazione: parte quindi con `CHANGEME` e con `REQUIRE_API_KEY=false`. Finché non lo avete verificato, usate le variabili nella finestra, come sopra, e controllate che abbiano effetto: il pannello deve chiedere la vostra password e una richiesta allo strato 3 senza chiave deve essere rifiutata. La cartella dei dati (database, registri, copie) è `%USERPROFILE%\.omniroute`.

Il file `.env` contiene segreti e non va mai inserito in un repository di codice.

Avvio e primo accesso

Con le variabili impostate, `omniroute serve` avvia il programma sulla porta 20128. Windows può mostrare un avviso del firewall: consentite solo le reti private oppure annullate, se l'indirizzo è 127.0.0.1. Il testo esatto dell'avviso è da verificare a schermo.

Aprite nel browser `http://localhost:20128`. Il pannello chiede la password. Se non l'avete impostata, parte la procedura guidata che ve la fa creare.

Per cambiare la password in seguito: nel pannello, Impostazioni, sezione Sicurezza; oppure da terminale `omniroute reset-password`.

Strato 1 superato quando il pannello si apre e accetta la password.

Parte 3. Collegare un fornitore con una chiave API

Un fornitore è l'azienda che mette a disposizione i modelli. In OmniRoute un collegamento si aggiunge dal pannello, nella sezione dei fornitori (i nomi esatti dei menu sono da verificare a schermo: cambiano tra le versioni). L'esempio è Anthropic:

1. Create la chiave nella console di Anthropic, in un'area di lavoro dedicata, con un tetto di spesa mensile. Il tetto vi protegge da un errore di configurazione.

2. Nel pannello di OmniRoute aggiungete un fornitore di tipo "chiave API", scegliete Anthropic e incollate la chiave.
3. Salvate e usate il pulsante di prova del collegamento, se c'è. Deve confermare che la chiave funziona.

La regola del corso sui fornitori. Si collegano solo fornitori usati tramite chiave API e con condizioni d'uso che escludono l'addestramento sui vostri dati. Non si collegano:

- account personali gratuiti (per esempio un account Google privato): le condizioni dei servizi gratuiti spesso permettono di usare i testi per migliorare il modello, e le risposte possono arrivare da lì senza che ve ne accorgiate;
- abbonamenti personali a un chatbot, usati al posto delle API: sono un altro prodotto, con altre regole, e possono violare le condizioni del servizio;
- servizi a pagamento inseriti come riserva silenziosa nella catena: costano senza che nessuno l'abbia deciso.

Le condizioni di ciascun fornitore si leggono sul suo sito e vanno ricontrollate: cambiano.

I collegamenti fatti con un accesso "tramite account" (e non con una chiave) hanno un token che scade. Quando scade, OmniRoute lo segnala con un errore (`invalid_grant`) e la combo perde tempo a provarlo prima di passare oltre. Un collegamento scaduto va riautorizzato o rimosso.

Parte 4. Creare una combo e una chiave OmniRoute

La combo

Una combo è una lista ordinata di modelli con una strategia. L'agente chiede "la combo X" e OmniRoute decide chi risponde davvero. Con la strategia a priorità (`priority`) si prova il primo; se fallisce o è saturo, si passa al secondo, e così via. OmniRoute offre anche altre strategie (per esempio `round-robin`, `cost-optimized`, `weighted`).

1. Nel pannello aprite la sezione Combo e createne una nuova.
2. Date un nome senza spazi, per esempio `Combo-Corso-Cascata`. Lo dovrete scrivere identico in OpenCode.
3. Aggiungete i modelli in ordine, dal primo che volete usare al meno preferito. Solo modelli dei fornitori collegati con chiave API.
4. Scegliete la strategia `priority`.

OmniRoute crea da solo anche combo "virtuali" con nomi come `auto/coding`, `auto/fast`, `auto/cheap`, costruite sui fornitori collegati. La documentazione di OmniRoute cita anche un modello `auto` che smista tra i collegamenti idonei, ma nel caso reale da cui nasce questa guida `auto` non compariva nell'elenco dei modelli e OpenCode non funzionava (vedi più avanti). La regola sicura: usare solo un `id` che compare nell'elenco.

La chiave OmniRoute

Con `REQUIRE_API_KEY=true` l'agente ha bisogno di una chiave OmniRoute. Va distinta dalla chiave del fornitore: la chiave OmniRoute dice a OmniRoute "questa richiesta è autorizzata", quella del fornitore resta dentro OmniRoute.

1. Aprite nel pannello la pagina Endpoints (la documentazione la indica come il posto da cui copiare la chiave).
2. Create una chiave, con un'etichetta che dica a cosa serve, per esempio `opencode-pc-casa`.
3. Copiatela una volta e custoditela: la usate al passo successivo.

Salvatela in una variabile d'ambiente dell'utente, così non finisce in nessun file di progetto:

```
setx OMNIROUTE_API_KEY "<chiave>"
```

`setx` vale per le finestre aperte da ora in poi, non per quella corrente. Chiudete PowerShell e riapritelo. Le applicazioni già aperte (compreso OpenCode desktop) vanno riavviate per vedere la variabile. Per usarla subito nella finestra attuale: `$env:OMNIROUTE_API_KEY = "<chiave>"`.

Parte 5. Provare OmniRoute da solo, prima di OpenCode

Qui sta il metodo che fa risparmiare ore: si prova ogni strato separatamente, dal basso. OpenCode viene per ultimo.

Strato 1: il pannello risponde. Già visto: `http://localhost:20128` nel browser.

Strato 2: i modelli disponibili. In PowerShell, con la chiave nella variabile:

```
curl.exe http://localhost:20128/v1/models -H "Authorization: Bearer
$env:OMNIROUTE_API_KEY"
```

Scrivete `curl.exe` con il `.exe`: in Windows PowerShell la parola `curl` da sola è un altro comando, con parametri diversi. La risposta è un elenco JSON. Cercate il campo `id` della vostra combo: quella stringa è il nome esatto del modello, da copiare tale e quale (maiuscole, trattini, barre comprese). L'elenco contiene anche le combo `auto/...` e i modelli dei fornitori collegati.

Strato 3: una richiesta vera. Create un file `richiesta.json` con questo contenuto, sostituendo il nome della combo con quello che avete copiato:

```
{
  "model": "Combo-Corso-Cascata",
  "messages": [{ "role": "user", "content": "Rispondi solo: ok" }]
}
```

Poi:

```
curl.exe -i http://localhost:20128/v1/chat/completions -H "Authorization: Bearer
$env:OMNIROUTE_API_KEY" -H "Content-Type: application/json" -d "@richiesta.json"
```

L'opzione `-i` mostra anche le intestazioni della risposta (gli "header"). Cercate:

- la prima riga, `HTTP/1.1 200 OK`: la richiesta è andata a buon fine;
- `x-omniroute-provider` e `x-omniroute-model`: dicono quale fornitore e quale modello hanno risposto davvero. Sono state osservate sulla versione 3.8.52; la documentazione consultata cita un'intestazione simile, `X-OmniRoute-Decision`. Se non le vedete, cercate quelle che iniziano per `x-omniroute-`.

Controllate che il fornitore che ha risposto sia quello che vi aspettate. Se risponde un collegamento diverso dal primo della combo, la combo ha saltato i precedenti: guardate perché (token scaduto, limite raggiunto, modello non disponibile).

Strato 4: OpenCode. Solo quando i tre strati precedenti funzionano.

Parte 6. Installare OpenCode

La documentazione di OpenCode consiglia, su Windows, di usare WSL (Windows Subsystem for Linux) per l'esperienza migliore, ma elenca anche le installazioni native. Per questa guida usiamo quelle native.

Da terminale (npm):

```
npm install -g opencode-ai
```

Altre vie indicate dalla documentazione: `choco install opencode` (Chocolatey), `scoop install opencode` (Scoop), oppure il file eseguibile dalla pagina delle versioni su GitHub. L'installazione con Bun su Windows non è ancora supportata.

Applicazione desktop. OpenCode esiste anche come app con finestra. La pagina di documentazione letta non riporta i passi di installazione: scaricatela da opencode.ai e controllate a schermo la procedura.

Una precauzione sulle versioni. Esistono due generazioni di OpenCode, la 1 e la 2, che usano lo stesso comando `opencode` e non possono convivere. L'installazione con npm porta la 1 (al 28 ottobre 2026 la versione pubblicata è la 1.19.5). Questa guida descrive la 1: se usate la 2, controllate i nomi dei file e dei campi nella sua documentazione.

Per controllare: `opencode --version`.

Parte 7. Il file `opencode.json`

OpenCode legge la sua configurazione da più posti e li fonde. In ordine di importanza crescente:

1. la configurazione globale dell'utente: `%USERPROFILE%\.config\opencode\opencode.json`;
2. la configurazione indicata dalla variabile `OPENCODE_CONFIG`, se c'è;
3. il file `opencode.json` nella cartella del progetto;
4. le cartelle `.opencode` del progetto.

Se la stessa impostazione compare in due posti vince quella più in basso nell'elenco. La cartella `.config` si crea a mano se manca.

Questo è il file completo per collegare OpenCode a OmniRoute:

```
{
  "$schema": "https://opencode.ai/config.json",
  "provider": {
    "omniroute": {
      "npm": "@ai-sdk/openai-compatible",
      "name": "OmniRoute",
      "options": {
        "baseURL": "http://localhost:20128/v1",
        "apiKey": "{env:OMNIROUTE_API_KEY}"
      },
      "models": {
        "Combo-Corso-Cascata": {
          "name": "Combo del corso (cascata)"
        }
      }
    }
  },
  "model": "omniroute/Combo-Corso-Cascata"
}
```

Riga per riga (il formato JSON non ammette commenti dentro il file, quindi le spiegazioni stanno qui):

Riga	Cosa significa
`"$schema"`	Indirizzo che descrive i campi ammessi. Serve all'editor per segnalare errori e suggerire i nomi dei campi. OpenCode non ne ha bisogno per funzionare.
`"provider"`	Elenco dei fornitori che OpenCode conosce, oltre a quelli già incorporati. Qui ne aggiungiamo uno.
`"omniroute"`	Il nome interno che scegliete voi per questo fornitore. Ricomparirà davanti al modello (`omniroute/...`).
`"npm": "@ai-sdk/openai-compatible"`	Il componente software che sa parlare con il servizio. Il campo si chiama `npm` , non `npmPackage` : un nome sbagliato viene ignorato senza avvisi. Per un servizio compatibile con l'interfaccia di OpenAI, come OmniRoute, il componente giusto è `@ai-sdk/openai-compatible` . Il componente `@ai-sdk/openai` serve per i servizi che offrono l'interfaccia `/v1/responses` .

Riga	Cosa significa
`"name"`	Il nome che vedete nei menu di OpenCode. Libero.
`"options"`	Le opzioni di collegamento.
`"baseURL"`	L'indirizzo di OmniRoute, con `/v1` in fondo.
`"apiKey": "{env:OMNIROUTE_API_KEY}"`	La chiave OmniRoute. La forma `{env:NOME}` dice a OpenCode di leggere la variabile d'ambiente `NOME` all'avvio. Se la variabile non esiste, il valore resta vuoto e le richieste vengono rifiutate.
`"models"`	I modelli di questo fornitore. Per un fornitore aggiunto a mano OpenCode non li scopre da solo: li dovete elencare voi.
`"Combo-Corso-Cascata"`	La chiave deve essere identica, carattere per carattere, al campo `id` della risposta di `/v1/models`. È il nome che OpenCode manda a OmniRoute.
`"name"` (dentro il modello)	Nome leggibile nei menu. Libero.
`"model"`	Il modello predefinito, nella forma `fornitore/modello`. Così OpenCode parte già con quello giusto. Da verificare a schermo che compaia selezionato con il comando `/models`.

Dove tenere la chiave. La forma `{env:NOME}` è sempre la scelta preferibile. Scrivere la chiave direttamente nel file (`"apiKey": "<chiave>"`) è accettabile in un solo caso: il file è quello globale nella vostra cartella utente (`%USERPROFILE%\.config\opencode\`), che non fa parte di nessun repository e non viene condiviso. Nel file `opencode.json` dentro un progetto la chiave non va mai: un repository si può pubblicare per distrazione e un segreto pubblicato va considerato perso, da revocare subito. In alternativa a `{env:...}` esiste `{file:percorso}`, che legge la chiave da un file a parte.

Parte 8. Primo avvio di OpenCode

Aprite un terminale nella cartella di un progetto (anche una cartella vuota di prova) e avviate `opencode`.

- Con `/models` si vede l'elenco dei modelli e si sceglie quello di OmniRoute.
- Scrivete una richiesta semplice, per esempio "elenca i file di questa cartella". Se risponde, tutta la catena funziona.

- Con `/init` OpenCode analizza il progetto e crea nella cartella il file `AGENTS.md`: le istruzioni permanenti del progetto (come si avvia, come si provano le modifiche, convenzioni). Si rilegge, si corregge a mano e si tiene nel repository. Se nella cartella c'è già un `CLAUDE.md`, OpenCode lo usa quando manca `AGENTS.md`; se ci sono entrambi vince `AGENTS.md`.
- Con il tasto Tab si passa da una modalità all'altra: `build` ha tutti gli strumenti e può modificare i file; `plan` serve a ragionare e proporre senza toccare il codice.
- I permessi si regolano nel campo `permission` di `opencode.json`. Ogni azione può essere `allow` (si esegue), `ask` (chiede conferma) o `deny` (vietata). Esempio prudente per i comandi del terminale:

```
{
  "permission": {
    "bash": {
      "*": "ask",
      "git status": "allow",
      "rm *": "deny"
    }
  }
}
```

Tra le regole che corrispondono vince l'ultima. Per default quasi tutto è permesso senza chiedere: se non volete sorprese, mettete `ask` come regola generale.

Errori frequenti

> Se state per fare l'attività "Trova l'errore", leggete questa sezione dopo: contiene le risposte.

Casi reali, nell'ordine in cui conviene controllarli (dal basso: servizio, modelli, richiesta, agente).

1. OpenCode non riceve risposta

- Sintomo: errore di connessione a `localhost:20128`. Con curl: `Failed to connect to localhost port 20128`. Il pannello nel browser non si apre.
- Causa: OmniRoute è spento. Se è stato avviato con `npm run dev` (installazione da sorgente), vive solo finché il terminale resta aperto. Dopo un riavvio del PC o una

finestra chiusa, nessuno ascolta sulla porta. I dati (fornitori, combo, chiavi) restano al loro posto, in `%USERPROFILE%\.omniroute`.

- Correzione: riavviare OmniRoute (`omniroute serve`, oppure `npm run dev` nella sua cartella) e ripetere lo strato 1. Per non ricadere nel problema, preferite l'installazione con npm.

2. Il campo del fornitore viene ignorato

- Sintomo: OpenCode non mostra il fornitore, o ne usa uno diverso da quello scritto, senza nessun messaggio.
- Causa: il campo è stato scritto `npmPackage`. Il nome giusto è `npm`. I campi sconosciuti vengono ignorati in silenzio.
- Correzione: rinominarlo in `npm`. Il `$schema` in cima al file aiuta l'editor a sottolineare i nomi sbagliati.

3. Il componente sbagliato

- Sintomo: errori strani sul formato della richiesta o della risposta, anche con OmniRoute acceso.
- Causa: `"npm": "@ai-sdk/openai"`. Quel componente è pensato per i servizi che offrono l'interfaccia `/v1/responses`. Con `@ai-sdk/openai-compatible` OpenCode usa invece `/v1/chat/completions`, la stessa interfaccia provata allo strato 3. OmniRoute espone anche `/v1/responses`, ma nel caso reale la configurazione con `@ai-sdk/openai` non funzionava.
- Correzione: `"npm": "@ai-sdk/openai-compatible"`.

4. Il nome del modello non esiste

- Sintomo: errore "modello non trovato", o il modello non compare in `/models`.
- Causa: nel campo `models` c'è un nome che non compare nell'elenco (nel caso reale `auto`) invece dell'`id` esatto restituito da `GET /v1/models` (per esempio `auto/coding`, oppure il nome della vostra combo). OpenCode non scopre i modelli di un fornitore aggiunto a mano.
- Correzione: copiare l'`id` dalla risposta dello strato 2 e usarlo come chiave dentro `models` e dopo `omniroute/` nel campo `model`.

5. Il PC non vede la chiave

- Sintomo: errore 401 (non autorizzato) pur avendo creato la chiave.

- Causa: la variabile `OMNIROUTE_API_KEY` è stata creata con `setx` ma OpenCode, o il terminale, erano già aperti. Oppure ha un nome diverso da quello scritto nel file.
- Correzione: chiudere e riaprire il terminale (e l'app), controllare con `echo $env:OMNIROUTE_API_KEY` che il valore ci sia, verificare il nome.

6. Risponde un fornitore diverso da quello creduto

- Sintomo: tutto funziona, ma `x-omniroute-provider` indica un collegamento che non pensavate di usare.
- Causa: il primo anello della combo era un collegamento non adatto (per esempio un account gratuito personale) o fuori servizio, e la cascata ha proseguito.
- Correzione: rivedere l'ordine della combo e togliere i collegamenti che non rispettano la regola del corso. Ripetere lo strato 3 dopo ogni modifica.

7. Un collegamento scaduto rallenta tutto

- Sintomo: le risposte arrivano, ma con ritardo; nel pannello compare `invalid_grant`.
- Causa: il token di un collegamento "tramite account" è scaduto e la cascata lo prova a vuoto.
- Correzione: riautorizzarlo o toglierlo dalla combo.

8. Impostazioni di sicurezza lasciate come da fabbrica

- Sintomo: nessuno. Per questo è l'errore più pericoloso.
- Causa: password del pannello rimasta `CHANGEME`, `REQUIRE_API_KEY` non impostata, OmniRoute in ascolto su tutta la rete.
- Correzione: parte 2 di questa guida. Cambiare la password, impostare `REQUIRE_API_KEY=true`, limitare l'indirizzo a 127.0.0.1.

Spegnere e riavviare

- **OmniRoute (npm):** `Ctrl+C` nella finestra in cui gira. Per riavviarlo, `omniroute serve` con le stesse variabili. Non è un servizio di Windows: se chiudete la finestra o riavviate il PC, resta spento finché non lo riavviate voi.
- **OmniRoute (Docker):** `docker stop omniroute` per fermarlo, `docker start omniroute` per riaccenderlo. Con l'opzione `--restart unless-stopped` riparte insieme a Docker.

- **OpenCode:** `/exit` nel programma, oppure `Ctrl+C`.
- **Dopo un riavvio del PC** controllate sempre lo strato 1 prima di accusare OpenCode.

Sicurezza, in sintesi

- Le chiavi (del fornitore e di OmniRoute) non entrano mai in un repository, in una chat o in uno screenshot. Se ne è uscita una, si revoca e se ne crea un'altra.
- Il file `opencode.json` di un progetto usa sempre `{env:NOME}`. La chiave scritta nel file è tollerata solo nel file globale nella cartella utente.
- Il file `.env` di OmniRoute resta fuori da Git: in un progetto con Git si controlla `.gitignore`.
- Sul fornitore si imposta un tetto di spesa mensile, in un'area di lavoro dedicata.
- OmniRoute ascolta solo su 127.0.0.1, con `REQUIRE_API_KEY=true` e una password vostra.
- Si collegano solo fornitori in chiave API con condizioni che escludono l'addestramento sui dati.
- L'agente può eseguire comandi sul vostro PC: tenete `ask` come regola generale e leggete ogni conferma prima di accettarla.
- I dati di studenti e famiglie non si usano in prove e dimostrazioni.

Lo schema in una riga

OpenCode (agente) → `http://localhost:20128/v1` (OmniRoute: chiave, combo, registri) → fornitore con chiave API (modello).
